

Лабораторная работа №2

Студент: Мансуров Роман

Группа: 6312-100503D

1. Цель работы

Изучение алгоритма умножения квадратных матриц и методов его распараллеливания с использованием технологии OpenMP. Исследование зависимости времени выполнения от размера матриц и количества вычислительных потоков. Оценка эффективности масштабирования алгоритма на многоядерной системе и получение статистических характеристик времени выполнения.

2. Описание работы алгоритма

В данной работе реализован классический алгоритм умножения двух квадратных матриц с применением многопоточности. Для матриц A и B размера $n \times n$ результирующая матрица C вычисляется по стандартной формуле, но выполнение внешнего цикла (по строкам матрицы A) распределяется между потоками.

Программа на C++:

1. Подключает заголовочный файл `<omp.h>` для управления потоками.
2. Принимает количество потоков как аргумент командной строки и устанавливает его через `omp_set_num_threads()`.
3. Читает входные матрицы, выполняет параллельное умножение и замеряет время работы именно вычислительной части.
4. Сохраняет результат для последующей верификации.

Python-скрипт автоматизирует эксперимент:

1. Генерирует случайные матрицы для размеров 200, 400, 800, 1200, 1600, 2000.
2. Для каждого размера запускает C++ программу сериями по 5 раз с разным количеством потоков (1, 2, 4, 8, 16).
3. Верифицирует результаты каждой итерации с помощью библиотеки NumPy.
4. Рассчитывает среднее время, доверительные интервалы и строит сравнительные графики производительности.

3. Характеристики вычислительной системы

Процессор:

- Модель: AMD Ryzen 7 2700 Eight-Core Processor
- Количество ядер: 8
- Тактовая частота: 3.20 GHz

Оперативная память:

- Объем: 24 Гб
- Тип: DDR4

Операционная система:

- Название: Windows 11
- Разрядность: 64bit

4. Результаты экспериментов

Размер 200x200	Размер 400x400	Размер 800x800	Размер 1200x1200
Потоков: 1	Потоков: 1	Потоков: 1	Потоков: 1
Запуск 1/5: 0.0040 сек	Запуск 1/5: 0.0465 сек	Запуск 1/5: 0.4339 сек	Запуск 1/5: 1.5808 сек
Запуск 2/5: 0.0040 сек	Запуск 2/5: 0.0474 сек	Запуск 2/5: 0.4251 сек	Запуск 2/5: 1.6626 сек
Запуск 3/5: 0.0040 сек	Запуск 3/5: 0.0464 сек	Запуск 3/5: 0.4336 сек	Запуск 3/5: 1.6487 сек
Запуск 4/5: 0.0040 сек	Запуск 4/5: 0.0467 сек	Запуск 4/5: 0.4309 сек	Запуск 4/5: 1.6203 сек
Запуск 5/5: 0.0050 сек	Запуск 5/5: 0.0465 сек	Запуск 5/5: 0.4238 сек	Запуск 5/5: 1.6315 сек
верификация пройдена	верификация пройдена	верификация пройдена	верификация пройдена
Потоков: 2	Потоков: 2	Потоков: 2	Потоков: 2
Запуск 1/5: 0.0020 сек	Запуск 1/5: 0.0250 сек	Запуск 1/5: 0.2402 сек	Запуск 1/5: 0.8239 сек
Запуск 2/5: 0.0030 сек	Запуск 2/5: 0.0305 сек	Запуск 2/5: 0.2354 сек	Запуск 2/5: 0.7868 сек
Запуск 3/5: 0.0030 сек	Запуск 3/5: 0.0247 сек	Запуск 3/5: 0.2296 сек	Запуск 3/5: 0.8070 сек
Запуск 4/5: 0.0020 сек	Запуск 4/5: 0.0307 сек	Запуск 4/5: 0.2275 сек	Запуск 4/5: 0.8067 сек
Запуск 5/5: 0.0031 сек	Запуск 5/5: 0.0231 сек	Запуск 5/5: 0.2240 сек	Запуск 5/5: 0.8146 сек
верификация пройдена	верификация пройдена	верификация пройдена	верификация пройдена
Потоков: 4	Потоков: 4	Потоков: 4	Потоков: 4
Запуск 1/5: 0.0020 сек	Запуск 1/5: 0.0122 сек	Запуск 1/5: 0.1101 сек	Запуск 1/5: 0.3940 сек
Запуск 2/5: 0.0020 сек	Запуск 2/5: 0.0121 сек	Запуск 2/5: 0.1211 сек	Запуск 2/5: 0.4104 сек
Запуск 3/5: 0.0020 сек	Запуск 3/5: 0.0122 сек	Запуск 3/5: 0.1111 сек	Запуск 3/5: 0.4030 сек
Запуск 4/5: 0.0010 сек	Запуск 4/5: 0.0152 сек	Запуск 4/5: 0.1123 сек	Запуск 4/5: 0.4137 сек
Запуск 5/5: 0.0010 сек	Запуск 5/5: 0.0121 сек	Запуск 5/5: 0.1184 сек	Запуск 5/5: 0.4836 сек
верификация пройдена	верификация пройдена	верификация пройдена	верификация пройдена
Потоков: 8	Потоков: 8	Потоков: 8	Потоков: 8
Запуск 1/5: 0.0010 сек	Запуск 1/5: 0.0091 сек	Запуск 1/5: 0.0678 сек	Запуск 1/5: 0.2378 сек
Запуск 2/5: 0.0020 сек	Запуск 2/5: 0.0081 сек	Запуск 2/5: 0.0644 сек	Запуск 2/5: 0.2576 сек
Запуск 3/5: 0.0010 сек	Запуск 3/5: 0.0144 сек	Запуск 3/5: 0.0897 сек	Запуск 3/5: 0.2739 сек
Запуск 4/5: 0.0011 сек	Запуск 4/5: 0.0081 сек	Запуск 4/5: 0.0774 сек	Запуск 4/5: 0.2790 сек
Запуск 5/5: 0.0010 сек	Запуск 5/5: 0.0091 сек	Запуск 5/5: 0.1081 сек	Запуск 5/5: 0.2356 сек
верификация пройдена	верификация пройдена	верификация пройдена	верификация пройдена
Потоков: 16	Потоков: 16	Потоков: 16	Потоков: 16
Запуск 1/5: 0.0010 сек	Запуск 1/5: 0.0080 сек	Запуск 1/5: 0.0633 сек	Запуск 1/5: 0.5338 сек
Запуск 2/5: 0.0010 сек	Запуск 2/5: 0.0080 сек	Запуск 2/5: 0.0590 сек	Запуск 2/5: 0.5632 сек
Запуск 3/5: 0.0010 сек	Запуск 3/5: 0.0090 сек	Запуск 3/5: 0.0614 сек	Запуск 3/5: 0.5464 сек
Запуск 4/5: 0.0020 сек	Запуск 4/5: 0.0070 сек	Запуск 4/5: 0.0591 сек	Запуск 4/5: 0.5327 сек
Запуск 5/5: 0.0020 сек	Запуск 5/5: 0.0102 сек	Запуск 5/5: 0.0643 сек	Запуск 5/5: 0.5518 сек
верификация пройдена	верификация пройдена	верификация пройдена	верификация пройдена

Размер 1600x1600	Размер 2000x2000	РЕЗУЛЬТАТЫ
Потоков: 1	Потоков: 1	--- Для 1 потоков ---
Запуск 1/5: 9.7987 сек	Запуск 1/5: 26.5986 сек	Размер 200: 0.0042 сек (± 0.0006)
Запуск 2/5: 9.4975 сек	Запуск 2/5: 26.6738 сек	Размер 400: 0.0467 сек (± 0.0005)
Запуск 3/5: 9.8971 сек	Запуск 3/5: 26.3645 сек	Размер 800: 0.4295 сек (± 0.0059)
Запуск 4/5: 9.5347 сек	Запуск 4/5: 26.2978 сек	Размер 1200: 1.6288 сек (± 0.0389)
Запуск 5/5: 9.6482 сек	Запуск 5/5: 27.8607 сек	Размер 1600: 9.6752 сек (± 0.2120)
верификация пройдена	верификация пройдена	Размер 2000: 26.7591 сек (± 0.7890)
Потоков: 2	Потоков: 2	--- Для 2 потоков ---
Запуск 1/5: 4.7641 сек	Запуск 1/5: 13.4358 сек	Размер 200: 0.0026 сек (± 0.0007)
Запуск 2/5: 4.4592 сек	Запуск 2/5: 13.4746 сек	Размер 400: 0.0268 сек (± 0.0044)
Запуск 3/5: 4.3849 сек	Запуск 3/5: 13.5167 сек	Размер 800: 0.2313 сек (± 0.0080)
Запуск 4/5: 4.5083 сек	Запуск 4/5: 13.5583 сек	Размер 1200: 0.8078 сек (± 0.0170)
Запуск 5/5: 4.5160 сек	Запуск 5/5: 13.4534 сек	Размер 1600: 4.5265 сек (± 0.1772)
верификация пройдена	верификация пройдена	Размер 2000: 13.4878 сек (± 0.0617)
Потоков: 4	Потоков: 4	--- Для 4 потоков ---
Запуск 1/5: 2.1975 сек	Запуск 1/5: 7.3014 сек	Размер 200: 0.0016 сек (± 0.0007)
Запуск 2/5: 2.1864 сек	Запуск 2/5: 7.0238 сек	Размер 400: 0.0128 сек (± 0.0017)
Запуск 3/5: 2.2997 сек	Запуск 3/5: 6.9331 сек	Размер 800: 0.1146 сек (± 0.0060)
Запуск 4/5: 2.1610 сек	Запуск 4/5: 7.0347 сек	Размер 1200: 0.4209 сек (± 0.0445)
Запуск 5/5: 2.1261 сек	Запуск 5/5: 6.9296 сек	Размер 1600: 2.1941 сек (± 0.0808)
верификация пройдена	верификация пройдена	Размер 2000: 7.0445 сек (± 0.1885)
Потоков: 8	Потоков: 8	--- Для 8 потоков ---
Запуск 1/5: 1.2751 сек	Запуск 1/5: 4.5191 сек	Размер 200: 0.0012 сек (± 0.0005)
Запуск 2/5: 1.1329 сек	Запуск 2/5: 4.2152 сек	Размер 400: 0.0098 сек (± 0.0033)
Запуск 3/5: 1.0627 сек	Запуск 3/5: 4.2361 сек	Размер 800: 0.0815 сек (± 0.0222)
Запуск 4/5: 1.0925 сек	Запуск 4/5: 4.3739 сек	Размер 1200: 0.2568 сек (± 0.0248)
Запуск 5/5: 1.2168 сек	Запуск 5/5: 4.1007 сек	Размер 1600: 1.1560 сек (± 0.1095)
верификация пройдена	верификация пройдена	Размер 2000: 4.2890 сек (± 0.2001)
Потоков: 16	Потоков: 16	--- Для 16 потоков ---
Запуск 1/5: 1.7158 сек	Запуск 1/5: 3.6114 сек	Размер 200: 0.0014 сек (± 0.0007)
Запуск 2/5: 1.7123 сек	Запуск 2/5: 3.5535 сек	Размер 400: 0.0085 сек (± 0.0015)
Запуск 3/5: 1.7044 сек	Запуск 3/5: 3.6092 сек	Размер 800: 0.0614 сек (± 0.0030)
Запуск 4/5: 1.7160 сек	Запуск 4/5: 3.5944 сек	Размер 1200: 0.5456 сек (± 0.0159)
Запуск 5/5: 1.6800 сек	Запуск 5/5: 3.6698 сек	Размер 1600: 1.7057 сек (± 0.0188)
верификация пройдена	верификация пройдена	Размер 2000: 3.6077 сек (± 0.0519)

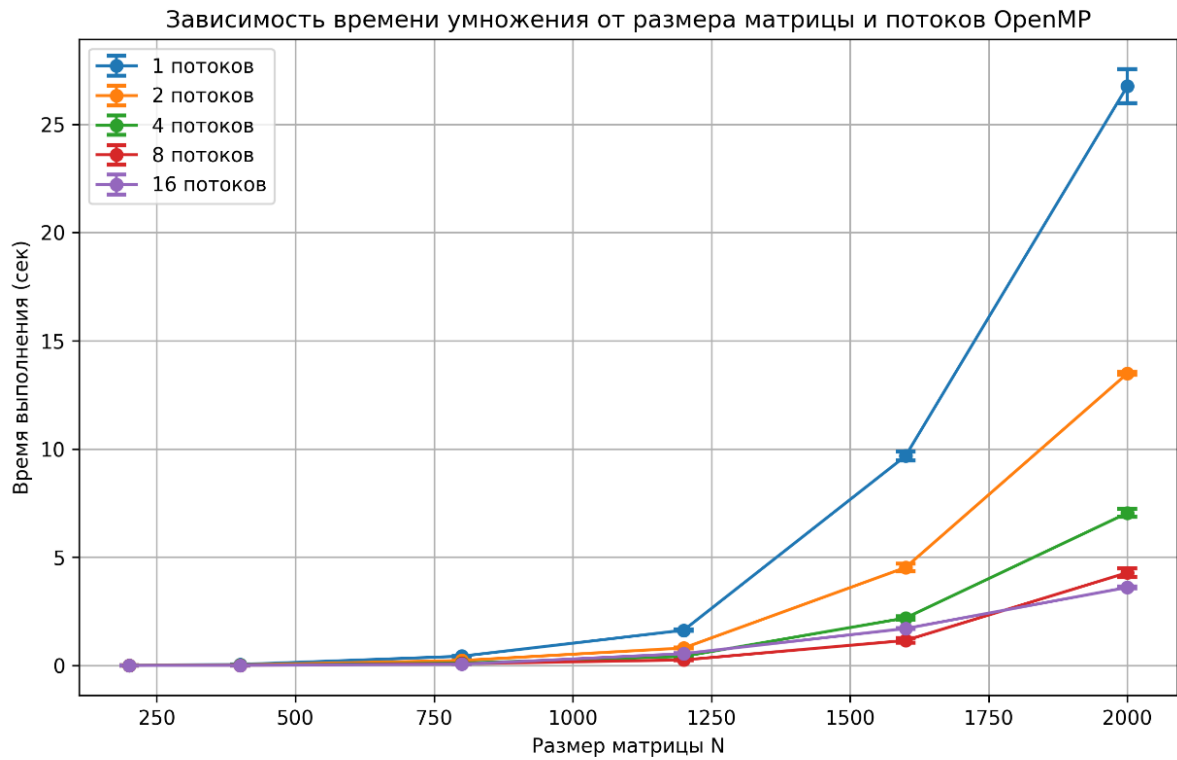
С увеличением количества потоков от 1 до 8 наблюдается почти линейное ускорение на больших матрицах (1600–2000). Например, для размера 2000 время сократилось с 26.76 сек (1 поток) до 4.29 сек (8 потоков), что подтверждает эффективное использование физических ядер процессора.

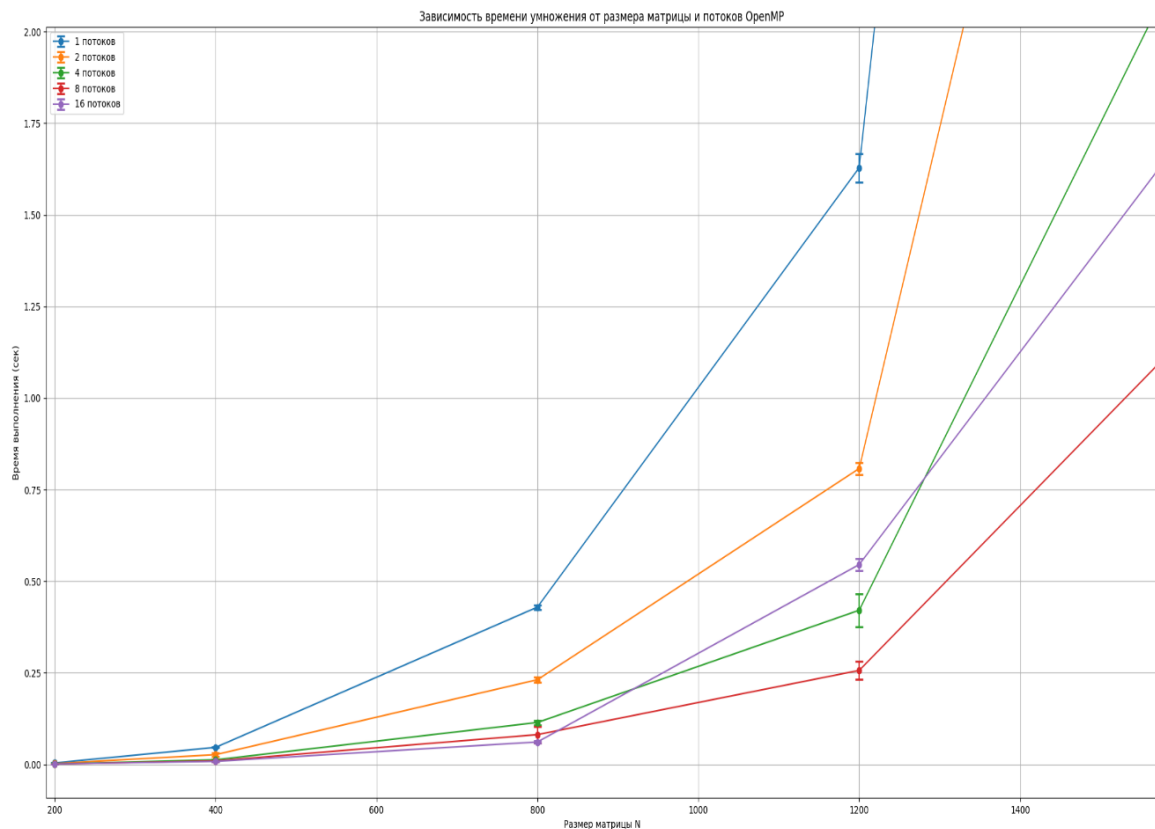
При переходе с 8 на 16 потоков прирост производительности замедляется, а на некоторых размерах (например, 1200 и 1600) время даже возрастает. Это объясняется тем, что 16 логических потоков делят ресурсы 8 физических ядер, а также возрастающими накладными расходами на синхронизацию кэш-памяти.

На малых размерах (200x200) выигрыш от распараллеливания минимален из-за того, что время на создание и управление потоками сопоставимо с временем самих вычислений.

Доверительные интервалы остаются узкими (в пределах 1–5% от среднего значения), что говорит о высокой стабильности работы параллельного алгоритма.

5. График





6. Вывод

В ходе лабораторной работы была разработана и исследована параллельная версия алгоритма умножения квадратных матриц с использованием OpenMP. Эксперименты подтвердили, что использование многопоточности позволяет значительно сократить время решения вычислительно трудоемких задач на многоядерных архитектурах.

Установлено, что для процессора AMD Ryzen 7 2700 оптимальное ускорение достигается при использовании 8 потоков (соответствует числу физических ядер). Дальнейшее увеличение количества потоков до 16 не дает пропорционального выигрыша из-за ограничений пропускной способности памяти. Разработанный комплекс из C++ программы и Python-скрипта позволил полностью автоматизировать процесс сбора данных и подтвердить корректность работы параллельного кода.

7. Исходный код

Исходный код представлен в репозитории на GitHub.